Design and Optimization of a 6 x 6 CMOS Multiplier

Xiaojun Zhou
12311815@mail.sustech.edu.cn

I. Introduction

Multiplication is one of the most important arithmetic operations in digital integrated circuits. It is widely used in arithmetic logic units, digital signal processors, image-processing hardware, and multiply-accumulate datapaths. At the transistor level, a multiplier is also a useful benchmark because its performance depends on both logic architecture and cell-level implementation.

A conventional array multiplier is attractive because of its regular structure and predictable routing. However, its long carry-propagation path can become the dominant delay source. For a 6 x 6 multiplier, this problem is already visible while the circuit remains small enough for complete schematic design, transistor sizing, layout, and post-layout verification.

The motivation of this project is to improve the delay-area product of a 6 x 6 CMOS multiplier through architecture selection, transistor-level logic optimization, and layout-aware design. The resulting circuit can be applied to compact arithmetic blocks in embedded processors, finite-impulse-response filters, small DSP accelerators, and other systems that require fast fixed-width multiplication.

II. Problem Formulation

The objective of this project is to design a 6 x 6 multiplier with low delay and small silicon area. According to the given performance metric, performance = 1 / (delay x area), the design must reduce both the critical-path delay and the number of transistors or layout footprint. The output loading condition is also considered, so the final gates must provide sufficient driving capability.

To achieve these objectives, the design is optimized in several steps. First, a baseline array multiplier is analyzed and compared with a carry-save architecture to reduce the critical-path length. Second, mirror-adder full adders, polarity alternation, and inverter elimination are used to lower transistor count and delay. Third, pseudo-NMOS logic, transistor sizing, output buffering, and compact layout placement are introduced to further reduce the delay-area product while preserving correct logic operation.

III. Design and Implementation

A. Design Methodology

A top-down design method is adopted in this work. The final optimization target is the delay-area product, so the design begins with the overall multiplier architecture. Several architectural iterations are used to reduce the critical-path length and the total transistor count. After the architecture is fixed, the corresponding logic cells are designed and further optimized in both structure and transistor usage.

According to the project performance rule, performance = 1 / (delay x area). Therefore, reducing the number of devices and shortening the critical path are the main objectives throughout the design. Power consumption is considered as a tradeoff, but it is not included directly in the performance metric.

B. Multiplier Architecture Optimization

1) Baseline Array Multiplier

The operating principle of an array multiplier is similar to manual multiplication. An N x N array multiplier uses N x N two-input AND gates to generate partial products, and N x (N - 1) adders accumulate the partial products and carries bit by bit. This structure is simple and regular, which is helpful for compact layout routing.

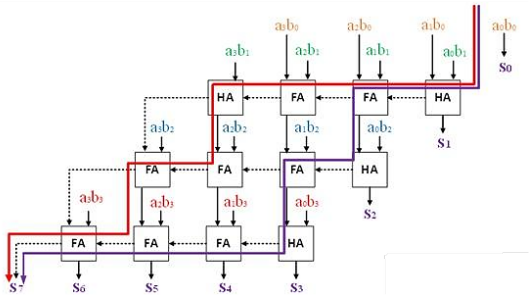


Fig. 1. Fig. 1. Baseline 6 x 6 array multiplier structure and critical path.

The main disadvantage of the array multiplier is its long critical path. For the 6 x 6 multiplier considered in this experiment, the critical path is estimated to pass through 6 + 2 + 2 + 2 + 2 = 14 adders. This path length directly limits the pre-layout delay.

2) Carry-Save Multiplier

To reduce the long carry-propagation path of the array multiplier, a carry-save multiplier is considered. The key difference is that carries are not propagated horizontally in the same row. Instead, each carry is saved and transferred diagonally to the next stage.

A full adder can be regarded as a 3:2 compressor. By applying a partial product to the Cin input of the first full-adder layer, the carry-save multiplier can use the same type and number of adder cells as the array multiplier while changing the carry-transfer pattern. For the 6 x 6 multiplier, the critical path is reduced to 6 + 1 + 1 + 1 + 1 = 10 adders, which significantly improves the expected pre-layout delay.

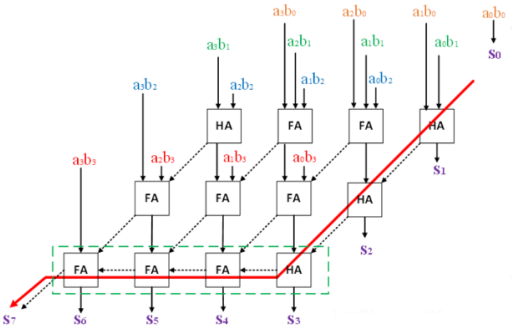


Fig. 2. Fig. 2. Carry-save multiplier structure and shortened critical path.

C. Logic Cells and Circuit-Level Optimization

1) Basic Gates and Adders

INV

Logic expression:

$$Y = A'$$

The inverter uses 2 transistors: one PMOS and one NMOS. The PMOS pulls the output up to VDD when A = 0, while the NMOS pulls the output down to GND when A = 1. It is used to generate complementary signals and to restore logic polarity.

NAND

Logic expression:

$$Y = (AB)'$$

A two-input NAND gate uses 4 transistors: two PMOS transistors in parallel and two NMOS transistors in series. The output becomes 0 only when both inputs are 1; otherwise, at least one PMOS provides a pull-up path and the output remains 1.

AND

Logic expression:

$$Y = AB$$

The AND gate can be built by connecting a NAND gate followed by an inverter: $Y = ((AB)')' = AB$. Therefore, it requires 6 transistors in total

XOR

Logic expression:

$$Y = A\,XOR\,B = AB' + A'B$$

The expression can be transformed as:

$$Y = AB' + A'B$$
$$Y = (A + B)(A' + B')$$
$$Y = (A + B)(AB)'$$

Here, (AB)' is treated as one complete intermediate signal and is generated by an independent NAND gate. By reusing this NAND output, the XOR implementation avoids generating both product terms separately. As a result, the transistor count can be reduced from 12 to 10. In this optimized XOR, the NAND part generates $(AB)'$, while the remaining transistors combine it with $(A + B)$ to produce the final XOR output.

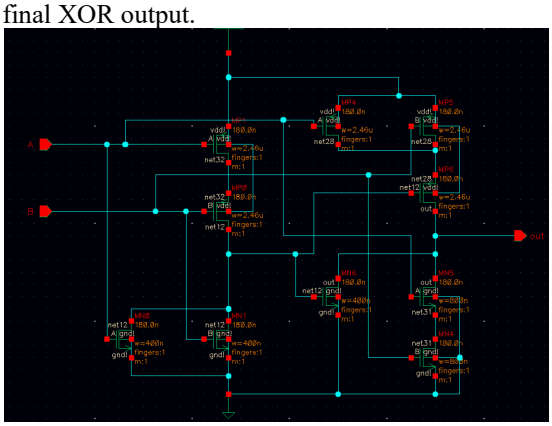


Fig. 3. Fig. 3. Transistor-level schematic of the optimized 10-transistor XOR gate.

HA

Logic expressions:

$$S = A\,XOR\,B$$

$$Cout = AB$$

The half adder is composed of one XOR gate for the sum output and one AND gate for the carry output. Using the optimized 10-transistor XOR and the 6-transistor AND, the HA requires 16 transistors in total. The XOR determines whether the two input bits are different, while the AND generates the carry when both inputs are 1.

FA

Logic expressions:

$$S = A\,XOR\,B\,XOR\,Cin$$
$$Cout = AB + ACin + BCin$$

Although the full adder can be constructed using basic logic gates, this design adopts the mirror-adder structure to further optimize transistor count and delay. The mirror adder has several advantages: its fully symmetric structure is convenient for later layout routing, it uses fewer series-connected transistors, and it provides lower delay.

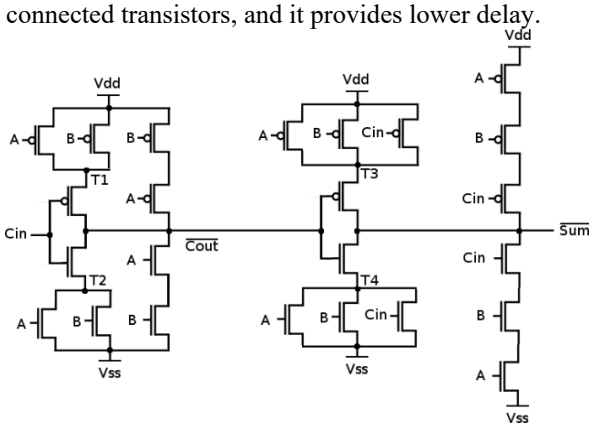
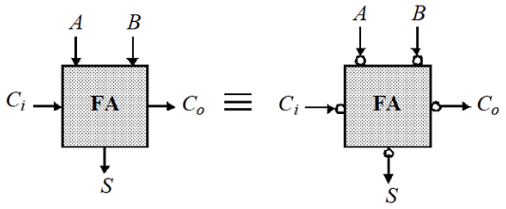


Fig. 4. Fig. 4. Mirror-adder full-adder schematic used for transistor-count and delay optimization.

2) Polarity Alternation and Inverter Reduction

From the logic units above, inverters appear in partial-product generation, in the HA carry output, and in the outputs of the FA. A useful property of the full adder is that when all of its inputs are inverted, all of its outputs are also inverted. Therefore, by adjusting input polarities and inter-cell connections, many explicit inverters can be removed.



$$\bar{S}(A,B,C_i) = S(\bar{A},\bar{B},\bar{C_i})$$
$$\overline{C_o}(A,B,C_i) = C_o(\bar{A},\bar{B},\bar{C_i})$$

Fig. 5. Fig. 5. Full-adder polarity property under simultaneous input inversion.

The half adder does not have the same complete inversion property as the full adder. However, the XOR sum output is unchanged when both inputs are inverted, and the HA sum does not need to be propagated further in this design. For the HA carry output in an inverted layer, the desired inverted result can be obtained by replacing the AND gate with a NOR gate. This also removes one inverter.

A	B	A'	B'	A XOR B	A' XOR B'
0	0	1	1	0	0
0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

Fig. 6. Fig. 6. Truth table showing that XOR is unchanged when both inputs are inverted.

A	B	A AND B	A'	B'	NOR(A',B')
0	0	0	1	1	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	1	0	0	1

Fig. 7. Fig. 7. Truth table showing that AND(A,B) is equivalent to NOR(A',B').

The carry-save multiplier is naturally suitable for this polarity-alternation method. In a conventional array multiplier, Cout is propagated horizontally, so inversion must be handled differently for odd and even adders within the same layer. The resulting connection pattern is more complicated. In the carry-save multiplier, both S and Cout are transferred to the next layer, so alternating the input polarity by layer is much cleaner.

The current multiplier contains five adder layers. If layers 1, 3, and 5 are selected as inverted-input layers, most partial-product AND gates can be replaced by NAND gates, saving the largest number of inverters. In the fifth layer, five additional inverters are still required because carries must be propagated horizontally, but this is a small cost compared with the total inverter reduction.

Replacing AND gates with NAND gates in the partial-product inputs saves 23 inverters, and the adder network saves another 30 inverters. Although the bottom carry-propagation row requires 5 additional inverters, the overall design still removes 48 inverters, equivalent to 96 transistors.

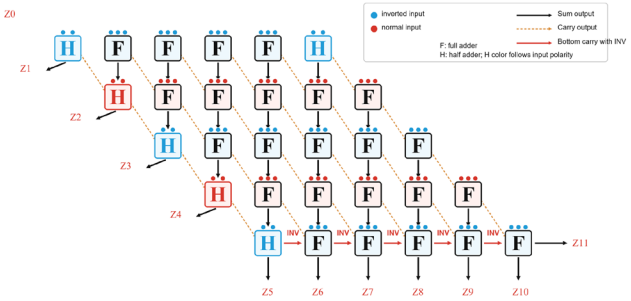


Fig. 8. Fig. 8. Polarity-alternated carry-save multiplier structure after inverter reduction.

3) Pseudo-NMOS Logic and Buffering

Since energy is not included in the project performance expression, pseudo-NMOS logic is an attractive choice. It uses a small number of PMOS transistors to replace a larger pull-up network, which can reduce transistor count and layout area while maintaining good delay after proper sizing. The main cost is higher static power consumption.

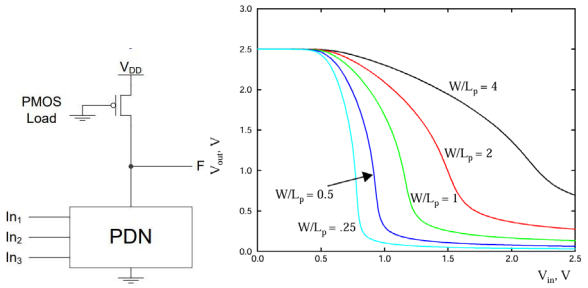


Fig. 9. Fig. 9. Pseudo-NMOS logic concept and inverter voltage-transfer curves under different PMOS sizing.

Pseudo-NMOS gates may also produce a relatively high V_{OL}. In addition, every output must drive a 50 fF load capacitance. For this reason, an output buffer is required. A two-stage buffer with a stage effort of 3 is selected to improve the output-driving capability.

D. Transistor Sizing and Simulation Plan

1) Sizing Strategy

The channel length is selected as the minimum 180 nm because a shorter channel reduces resistance, capacitance, delay, and area. Transistor width affects both capacitance and driving capability. Since the multiplier transistor count has already been strongly optimized, the minimum width of 400 nm is used as the starting point, and only necessary transistors are enlarged.

Simulation shows that when the NMOS width is 400 nm, choosing a PMOS width of 1.23 um can produce approximately symmetric rising and falling transitions. This sizing result is used as a reference for the final gate designs.

2) Delay Measurement and Load-Capacitance Effect

For delay measurement, the input signal uses rise and fall times of 0.2 ns, measured from 10% to 90% of the signal transition. Each basic gate is simulated under three extrinsic load capacitances: C1 = 15 fF, C2 = 8C1 = 120 fF, and C3 = 64C1 = 960 fF. The measured delays for the basic gates are summarized in Table 2.

The propagation delay increases as the extrinsic load capacitance increases because the output node requires more charge or discharge current. A first-order model can be written as $tp = k \cdot R_{eff}(C_{int} + C_L)$, where Reff is the effective pull-up or pull-down resistance, Cint is the internal parasitic capacitance, and CL is the external load capacitance. Therefore, increasing CL increases the RC time constant and produces a longer transition time.

For a CMOS inverter, the relationship between delay and load capacitance is relatively direct because there is only one PMOS pull-up path and one NMOS pull-down path. In more complex gates with multiple inputs, such as NAND, NOR, AND, and XOR, the effective resistance and internal capacitance depend on the conducting path and the input vector. Series transistors increase Reff, while additional diffusion nodes increase Cint. As a result, the delay-load slope becomes larger and the rise/fall delay asymmetry becomes more obvious than in a simple inverter.

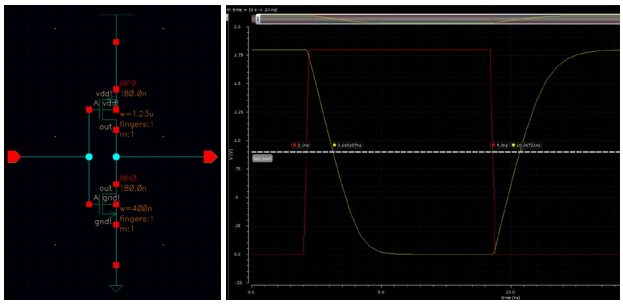


Fig. 10. Fig. 10. Inverter sizing simulation setup and transient response under capacitive loading.

3) Architecture-Level Comparison

A comparison table will be used to summarize the transistor count and delay of the original array multiplier, the carry-save multiplier, and the pseudo-NMOS optimized design. The numerical entries are reserved for the final simulation data.

	INV	NAND	AND	BUF	HA	FA	Total transistors	tp
Baseline	0	0	36×6	12×4	6×16	24×28	1032	2.0903
Carry-Save	5×2	23×4	13×6	12×4	6×14	24×24	888	1.0754
pseudo	5×2	23×3	13×5	12×4	6×10	24×14	588	0.6023

TABLE I. TABLE 1. ARCHITECTURE-LEVEL TRANSISTOR COUNT AND PRE-LAYOUT DELAY COMPARISON.

E. *Layout Design Considerations*

1) Cell Schematic and Layout

INV

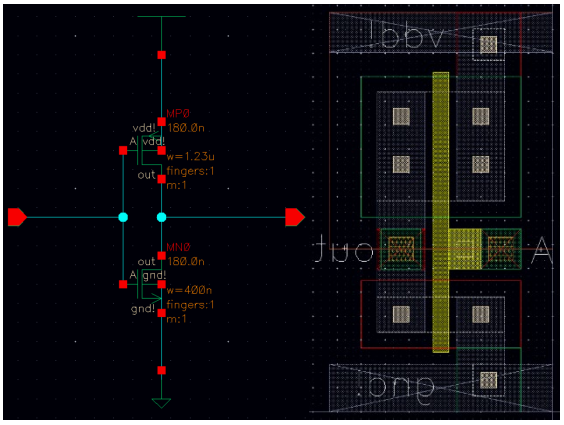


Fig. 11. Fig. 11. INV schematic and corresponding layout.

NAND

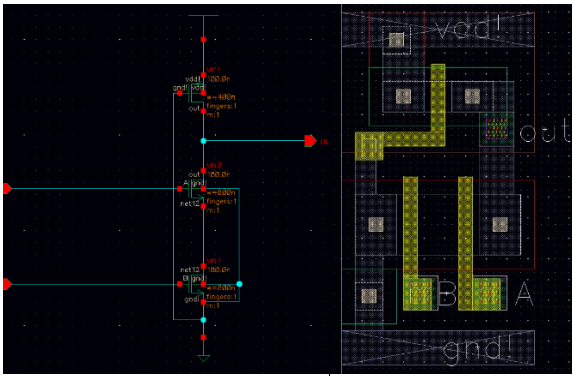


Fig. 12. Fig. 12. NAND gate schematic and corresponding layout.

NOR

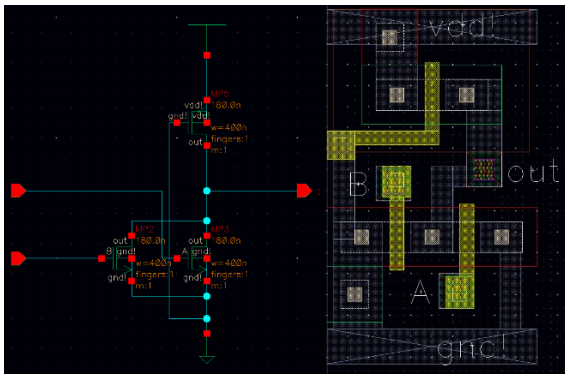


Fig. 13. Fig. 13. NOR gate schematic and corresponding layout.

AND

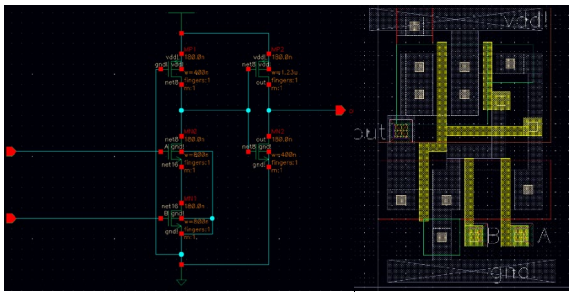


Fig. 14. Fig. 14. AND gate schematic and corresponding layout.

XOR

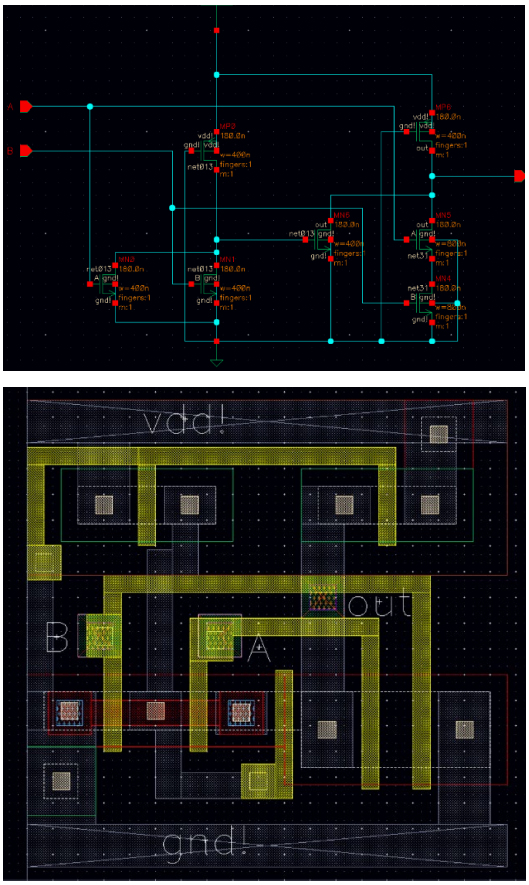


Fig. 15. Fig. 15. XOR gate schematic and corresponding layout.

Half Adder

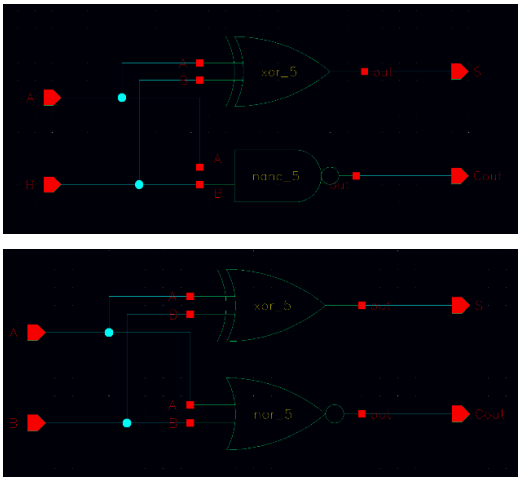


Fig. 16. Fig. 16. Half-adder logic implementations for normal and inverted-polarity layers.

Full Adder

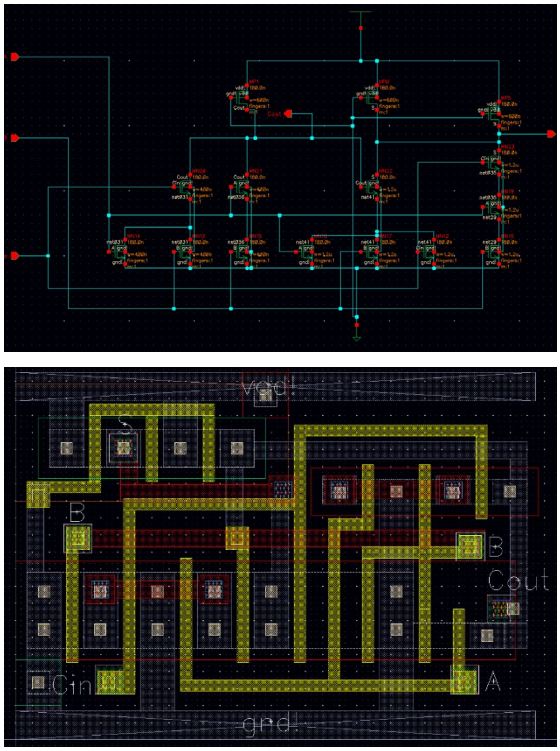


Fig. 17. Fig. 17. Mirror-adder full-adder schematic and corresponding layout.

Buffer

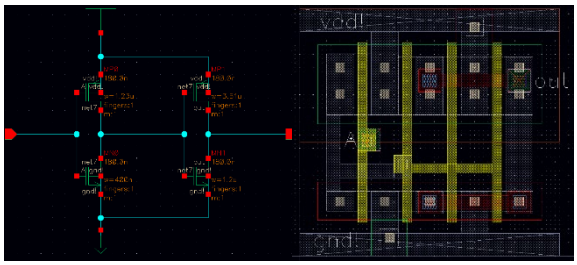


Fig. 18. Fig. 18. Buffer schematic and corresponding layo

TABLE II. TABLE 2. BASIC-GATE TRANSISTOR SIZING AND POST-LAYOUT DELAY UNDER DIFFERENT LOAD CAPACITANCES.

	NMOS W/L(μm)	PMOS W/L(μm)	C=15f $t_{pHL}/t_{pLH}(ns)$	C=120f $t_{pHL}/t_{pLH}(ns)$	C=960f $t_{pHL}/t_{pLH}(ns)$
INV	0.4/0.18	1.23/0.18	0.1037/0.1118	0.4552/0.4596	3.2275/3.2170
NAND	0.8/0.18	0.4/0.18	0.1650/0.1905	0.5720/0.5450	3.7755/3.3731
AND	0.4/0.18	0.4/0.18	0.1037/0.1573	0.4521/1.0044	3.2599/7.7400
NOR	0.4/0.18	0.4/0.18	0.1290/1.1739	0.5654/1.0800	2.8834/8.1680
XOR	0.4/0.18	0.4/0.18	0.2776/0.1103	1.1218/0.4623	7.8992/3.2473

TABLE III. TABLE 3. LAYOUT DIMENSIONS OF THE BASIC CELLS.

	INV	NAND	NOR	AND	XOR	BUF	FA
W(μm)	2.52	2.52	2.7	3.42	5.04	5.22	8.28
L(μm)	4.51	4.62	4.58	5.45	4.9	4.96	5.805

The following subsections summarize the placement, routing, and verification considerations for the layout implementation.

2) Placement and Area Reduction

Cell flipping is used to share power rails and reduce the area occupied by supply routing. Because different cells have different heights, several placement strategies are considered: arranging cells horizontally, merging multiple rows of narrow-input cells into one row, and splitting the half adder when it helps remove unnecessary whitespace.

3) Routing Strategy

When routing becomes difficult, multiple metal layers are used. A horizontal-even and vertical-odd routing convention is adopted, so adjacent routing layers have clear preferred directions and fewer conflicts.

4) Layout Verification

After layout extraction, the complete multiplier is verified through layout inspection and post-layout transient simulation. The final layout and waveform are used to evaluate whether the optimized architecture still satisfies the timing and area objective after parasitic effects are included.

The final complete multiplier layout occupies 50.04 um x 45.455 um, corresponding to an area of approximately 2274.57 um^2.

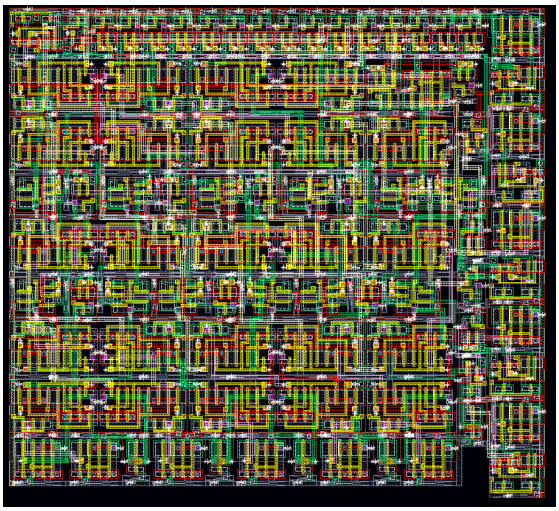


Fig. 19. Fig. 19. Complete layout of the optimized 6 x 6 multiplier.

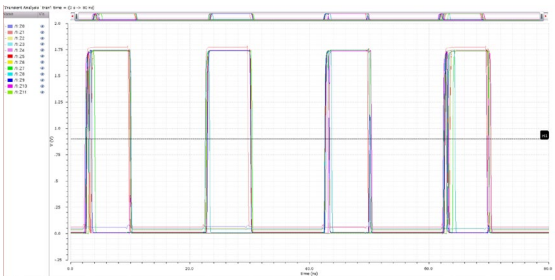


Fig. 20. Fig. 20. Post-layout transient simulation waveform of the complete multiplier.

TABLE IV. TABLE 4. FINAL MULTIPLIER PERFORMANCE SUMMARY.

	$t_p(ns)$
Schematic	0.8307
Post-layout	1.3864

IV. CONCLUSIONS

This project first optimized the top-level multiplier architecture. The baseline array multiplier was replaced by a carry-save structure to shorten the critical path, and the inverter-elimination method was coordinated with the carry-save data flow. By carefully arranging normal and inverted signal polarities between odd and even adder layers, most intermediate signals could be passed directly to the next layer without restoring their polarity at every stage.

The second major optimization was the transistor sizing of the pseudo-NMOS structure. The PMOS and NMOS width-to-length ratios had to be selected carefully. If the devices were too small, the effective resistance became too large and the delay increased. If the devices were made too large, the resistance ratio between the pull-up load and pull-down network could become inappropriate, causing an incorrect output voltage level or logic error. The final sizing was therefore chosen as a compromise between delay, logic swing, and area.

The most difficult part of the project was the final layout connection. Since the area of each logic cell was minimized as much as possible and the multiplier itself contains many closely coupled signal paths, routing the complete circuit became challenging. Five metal layers had to be used, and several unavoidable routing overlaps remained. These overlaps introduced additional parasitic capacitance and increased the post-layout delay compared with the schematic simulation.

Future improvement should focus on routing-aware cell design and global floorplanning. Leaving slightly more routing resources inside each cell, reducing long interconnects, and optimizing the relative placement of adder layers could reduce parasitic capacitance while keeping the delay-area product competitive.

ACKNOWLEDGMENT

I would like to thank myself for carrying this project through independently, for not giving up until the final moment, and for enduring the long nights required to finish the design. Every effort invested in this work was worthwhile.

REFERENCES

[1] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, Digital Integrated Circuits: A Design Perspective, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[2] "Pseudo-NMOS logic," Zhihu, [Online]. Available: https://zhuanlan.zhihu.com/p/586863445. Accessed: Jun. 8, 2026.

[3] “CMOS inverter and voltage transfer characteristics,” Zhihu, [Online]. Available: https://zhuanlan.zhihu.com/p/102387648. Accessed: Jun. 8, 2026.

[4] “CMOS basic logic gates,” Zhihu, [Online]. Available: https://zhuanlan.zhihu.com/p/119444283. Accessed: Jun. 8, 2026.

[5] “Mirror adder / full adder circuit design notes,” CSDN Blog, [Online]. Available: https://blog.csdn.net/weixin_45791458/article/details/133518505. Accessed: Jun. 8, 2026.

[6] “Layout design and CMOS process notes,” Zhihu, [Online]. Available: https://zhuanlan.zhihu.com/p/602684164. Accessed: Jun. 8, 2026.